VEK 0.4 / Wekretia

Game Development Documentation & Development Plan

Document type:	Technical and game-design project documentation
Project stage:	Early prototype / pre-production
Project type:	Story-driven 2D RPG
Engine:	Godot Engine
Programming language:	GDScript
Visual tools:	Krita, Aseprite
Version control:	Git and GitHub
Document language:	English
Status:	Active development

Document purpose

This document describes the current technical state, design direction, development principles and roadmap of VEK 0.4. It separates systems that already exist in the prototype from concepts that are still planned, experimental or subject to change.

VEK 0.4 is developed by Daniel Abramek as an independent game-development project. The game is set in Wekretia, an original fictional universe developed over many years.

Last updated: August 2026

Contents

1 Executive Summary

VEK 0.4 is an early-stage 2D role-playing game prototype developed in Godot Engine using GDScript. It is set in Wekretia, an original dark-fantasy universe that predates the game itself and has been developed through worldbuilding, stories, roleplay, characters, politics, religions, regions and conflicts.

The current version is intentionally focused on foundations rather than feature count. Development currently concentrates on player movement, directional animation, input handling, attack-state logic, scene organisation and early interface work.

The long-term direction is a story-driven top-down RPG with exploration, real-time combat, dialogue, quests, player choices, world interaction and strong atmosphere.

This document distinguishes between **implemented prototype functionality** and **planned game systems**. Planned systems are design targets and must not be treated as completed features.

Primary objective

Build a technically understandable and artistically coherent RPG prototype that can grow into a larger game without sacrificing maintainability, iteration speed or the identity of the Wekretia setting.

2 Project Goals

The project should gradually evolve toward a game that:

- turns the existing Wekretia universe into an explorable interactive setting;
- combines story, atmosphere, exploration and real-time gameplay;
- treats worldbuilding as part of gameplay rather than only background text;
- uses player choices to influence characters, quests and local outcomes;
- supports a readable and maintainable gameplay-code structure;
- remains technically realistic for an independent developer;
- grows through small tested systems rather than large unverified feature batches;
- develops a distinctive visual and narrative identity instead of copying one reference game;
- provides a practical long-term environment for learning game programming and design.

3 Project Background

3.1 Wekretia Before the Game

Wekretia existed before VEK 0.4.

The setting has been developed over many years through original worldbuilding, personal stories and roleplay. Characters, regions, political structures, beliefs and historical events were created gradually rather than as a single design document written for the game.

Some events that emerged during roleplay became part of the setting's canon. This gives the universe a history shaped both by authored design and by stories that developed through interaction.

3.2 General Setting Direction

Wekretia is designed as a dark, grotesque fantasy world with selected industrial influences.

One of its important motifs is dangerous crystal-based technology. Crystals may function as resources, sources of power or technological foundations, but their use can also carry serious consequences.

The intended tone combines:

- dark fantasy;
- grotesque imagery;
- social and political conflict;
- early industrial influences;
- dangerous crystal-based technology;
- regional cultures with distinct identities;
- a world that does not exist only to serve the player.

4 Evolution of the Project

4.1 Earlier Technical Direction

Before moving to Godot, the game concept was explored through programming experiments using Java and JavaFX.

That approach was eventually abandoned in favour of a dedicated game engine.

Around April 2026, the project moved toward:

Godot Engine + GDScript

This change allows the project to use a proper scene system, dedicated 2D tools, animation support, physics nodes and a workflow designed specifically for games.

4.2 Current Game Direction

The current direction is a top-down 2D RPG, potentially using a slightly isometric visual presentation in selected areas.

The project is intended to combine:

- exploration;
- real-time combat;
- narrative;
- dialogue;
- quests;
- worldbuilding;
- player choices and consequences;
- interaction with environments and characters.

Atmospheric inspiration can come from games such as *Darkwood*, but the project is not intended as a mechanical or visual copy of any one title.

5 Technical Architecture

5.1 Application Model

The current project uses a lightweight Godot architecture based on scenes, nodes and GDScript files.

Layer	Technology / responsibility
Game engine	Godot Engine
Programming	GDScript
Player body	`CharacterBody2D`
Character animation	`AnimatedSprite2D`
Input	Godot Input Map
2D artwork	Krita and Aseprite
Scene composition	Godot scene system
Version control	Git and GitHub

5.2 Architectural Principles

Development should follow several rules:

- **Small systems first:** implement one understandable gameplay system at a time.
- **Separate state from presentation:** gameplay state and animation state should not become one large conditional block.
- **Reusable scenes:** repeated game objects should eventually be represented as reusable scenes.
- **Readable scripts:** avoid premature abstraction, but also avoid large monolithic scripts.
- **No fake features:** documentation and repository descriptions should reflect the actual prototype.
- **Iterative refactoring:** working code may be reorganised after its behaviour is understood.
- **Engine-native solutions first:** prefer built-in Godot systems before introducing unnecessary external dependencies.

6 Current Prototype Structure

The exact project tree may change during development, but the repository currently follows a Godot-oriented structure containing gameplay scripts, scenes, assets and project files.

A future cleaned structure may evolve toward something similar to:

```
vek-0.4/
|
|-- assets/
| |-- characters/
| |-- environments/
| |-- ui/
| |-- audio/
| `-- fonts/
|
|-- scenes/
| |-- player/
| |-- world/
| |-- ui/
| |-- menus/
| `-- locations/
|
|-- scripts/
| |-- player/
| |-- combat/
| |-- ui/
| `-- systems/
|
|-- docs/
| |-- VEK_0.4_Project_Documentation_English.tex
| `-- VEK_0.4_Project_Documentation_English.pdf
|
|-- project.godot
|-- README.md
`-- .gitignore
```

The exact names are not requirements. The goal is to keep assets, scenes and gameplay logic understandable as the project grows.

7 Implemented Gameplay Foundation

Implemented or actively prototyped

The following functionality is part of the current VEK 0.4 prototype and can be treated as real development work.

7.1 Player Controller

The player is built around Godot's:

`CharacterBody2D`

The visual character representation uses:

`AnimatedSprite2D`

The controller currently handles four-direction movement and stores the direction in which the character most recently faced.

7.2 Input

Movement and attack behaviour use named Godot Input Map actions rather than direct hard-coded keyboard checks.

Conceptually, the project uses actions such as:

```
left_input
right_input
up_input
down_input
attack_input
```

This approach makes future control remapping or controller support easier than binding gameplay code directly to specific keys.

7.3 Movement

The player can move in four directions.

The movement system is based on a direction vector applied to the character velocity:

```
velocity = direction * SPEED
```

The exact movement speed remains a game-feel parameter and may change during tuning.

7.4 Direction Memory

The controller uses a value conceptually represented as:

```
last_direction
```

This preserves the player's facing direction after movement stops and allows the correct idle or attack animation to be selected.

8 Animation System

8.1 Directional Animation

The current prototype includes directional idle and movement animation logic.

Movement animation groups include:

- side movement;
- front movement;
- back movement.

Idle animations follow the same directional structure.

For horizontal movement, sprite flipping can be used to reduce unnecessary duplicate animation sets.

8.2 Attack Animation

Directional attack animation logic is also being prototyped.

The implementation needs to coordinate:

- current facing direction;
- attack state;
- attack duration;
- animation selection;
- movement blocking during an attack.

The system is still evolving and should not yet be treated as a finished combat animation framework.

9 Current Attack State

9.1 Basic Attack Logic

The prototype contains an initial attack-state implementation.

The controller uses state values conceptually represented as:

```
is_attacking
attack_timer
```

During the attack state, player movement is blocked.

This provides a foundation for more complete combat-state handling later.

9.2 Current Limitations

The following combat systems are **not complete**:

- production-ready hitbox and hurtbox system;
- complete enemy combat behaviour;
- weapon framework;
- equipment system;
- damage and defence balancing;
- advanced dodge mechanics;
- tactical aiming;
- finished combat feedback.

10 Scene and Interface Work

The repository also contains early work related to game scenes, menus and UI.

These elements may include:

- an intro or title flow;
- menu scenes;
- settings-related interface work;
- early in-game UI;
- location scenes such as an inn or other prototype spaces;
- music and audio assets used for testing.

These pieces are still under development and should not be interpreted as complete final systems.

11 Game Design Pillars

11.1 Exploration

Exploration should make the world feel larger than the immediate quest objective.

The player should be rewarded for paying attention to:

- environments;
- characters;
- regional culture;
- optional routes;
- local stories;
- environmental details.

11.2 Worldbuilding Through Play

Lore should not exist only as large blocks of explanatory text.

Whenever practical, information about the world should be communicated through:

- architecture;
- NPC behaviour;
- dialogue;
- environmental objects;
- quests;
- conflicts;
- visual differences between regions.

11.3 Consequences

Player decisions are intended to influence more than a final ending screen.

Possible future consequences may affect:

- NPC relationships;
- quest availability;
- local political situations;
- access to information;
- the fate of individual characters;
- the player's reputation;
- later narrative events.

12 Combat Design Direction

The desired combat direction is real-time and more deliberate than simply standing next to an enemy and repeatedly pressing one attack button.

Possible long-term elements include:

- directional attacks;
- multiple weapon types;
- armour;
- dodging;
- tactical positioning;
- environmental interactions;
- mouse-based aiming for selected actions.

The items above describe combat design direction. The current prototype only contains the early player movement, animation and attack-state foundation.

13 World Interaction

One long-term design goal is to make the environment more than static decoration.

Possible interactions include:

- objects reacting to damage;
- selected flammable environmental elements;
- using surroundings during combat;
- NPC responses to visible player actions;
- environmental states that change during quests or conflicts.

These features remain experimental design ideas and are not part of the current implemented prototype.

14 Narrative and Dialogue Direction

The game is intended to use dialogue as part of the larger gameplay system rather than as an isolated visual-novel layer.

A future dialogue system should be able to support:

- branching responses;
- conditional dialogue;
- quest state;
- relationship state;
- faction or reputation checks;
- consequences that appear later rather than immediately;
- optional lore and character information.

The exact dialogue data structure has not yet been finalised.

15 World Scope

Wekretia is larger than the realistic scope of one independent game project.

The project therefore should not attempt to simulate the entire universe at once.

A more practical target is:

Scope principle

Present a large and culturally rich world through a carefully directed journey across selected regions, locations and conflicts.

This allows the game to communicate the scale of Wekretia without requiring every region to be fully implemented.

16 Visual Direction

The current visual direction is based around 2D artwork and animation, with Krita and Aseprite used as important tools in the art workflow.

The game should eventually feel:

- dark;
- dirty;
- organic;
- grotesque;
- atmospheric;
- culturally varied between regions.

The visual identity should support the setting rather than imitate a generic fantasy asset pack.

16.1 Character and Environment Readability

Even with a dark palette, gameplay information must remain readable.

The final art direction should preserve clear distinctions between:

- player character;
- enemies;
- interactable objects;
- background decoration;
- hazards;
- UI elements.

17 Audio Direction

The prototype already uses audio assets during development.

Long-term audio work may include:

- ambient location audio;
- regional music themes;
- combat feedback;
- footsteps;
- environmental sounds;
- UI feedback;
- creature and NPC sounds.

Third-party audio must only be included publicly when its licence allows redistribution inside the repository or the asset is otherwise legally usable.

18 Repository and Asset Policy

Because VEK 0.4 is a public repository, project files should be reviewed before every major public update.

18.1 Source Files

The repository may contain:

- GDScript source code;
- Godot scene files;
- project configuration;
- original artwork;
- documentation;
- appropriately licensed third-party assets.

18.2 Files That Should Stay Excluded

Generated and local files should remain excluded through `.gitignore`, including items such as:

- Godot editor cache;
- generated import data;
- temporary files;
- editor-specific local settings;
- backup files;
- build output;
- secrets and environment files.

18.3 Third-Party Assets

Every public third-party asset should have a clear licence.

If the licence is unclear, the asset should not be treated as safe for redistribution.

19 Development Method

The project follows an iterative learning workflow.

A typical development cycle is:

1. choose one small gameplay problem;
2. implement a minimal working version;
3. test behaviour in the engine;
4. identify technical problems;
5. debug the system;
6. simplify or refactor where useful;
7. document what is actually working;
8. move to the next system.

This process deliberately prioritises understanding over rapidly accumulating features.

20 Testing Strategy

20.1 Gameplay Testing

Current and future gameplay tests should verify:

- movement in all intended directions;
- correct idle animation after movement;
- correct directional animation selection;
- attack-state entry and exit;
- blocked movement during attack when intended;
- no permanent lock after an interrupted animation;
- correct behaviour after repeated input;
- stable scene changes.

20.2 Regression Testing

After changing player or animation code, earlier behaviour should be checked again.

For example:

Change	Regression check
Attack logic modified	Verify movement and idle still work
Animation names changed	Verify all directional states resolve correctly
Input Map changed	Verify movement and attack actions still fire
Scene hierarchy changed	Verify node references remain valid
Player script refactored	Verify attack lock and direction memory still work

21 Debugging Principles

Debugging should aim to identify the state that caused incorrect behaviour rather than only patch the visible symptom.

Useful debugging questions include:

- What state is the player currently in?
- What direction is stored in `last_direction`?
- Which animation name is being requested?
- Is `is_attacking` being cleared correctly?
- Is the timer reaching the expected value?
- Is an animation being restarted every frame?
- Is a node path still valid after scene changes?

22 Implemented Features vs. Planned Systems

22.1 Implemented / Prototyped

Current repository scope

- Godot project setup;
- GDScript gameplay code;
- `CharacterBody2D` player;
- `AnimatedSprite2D` character animation;
- custom Input Map actions;
- four-direction movement;
- `last_direction` tracking;
- directional idle and movement animation logic;
- early directional attack animation logic;
- `is_attacking` state;
- attack timer;
- movement blocking during attack;
- early menu and UI work;
- early scene and location prototypes.

22.2 Planned / Not Yet Complete

- full dialogue system;
- full quest system;
- factions and reputation;
- inventory;
- equipment;
- character progression;
- advanced enemy AI;
- complete hitbox / hurtbox architecture;
- multiple weapon systems;
- larger region structure;
- save/load;
- complex narrative consequence system;
- fully interactive environments;
- production-ready combat balancing.

23 Development Roadmap

23.1 Phase 1 — Player Foundation

In progress

- four-direction movement;
- direction memory;
- idle and run animation states;
- attack-state foundation;
- improve code readability;
- remove animation-state bugs;
- stabilise current player behaviour.

23.2 Phase 2 — Combat Prototype

- define hitbox and hurtbox structure;
- create one test enemy;
- add damage exchange;
- add health state;
- improve attack timing;
- add basic combat feedback;
- test one complete player-versus-enemy interaction.

23.3 Phase 3 — World Interaction Prototype

- create reusable interactable objects;
- create one small playable location;
- add basic NPC interaction;
- prototype object interaction;
- test transitions between locations.

23.4 Phase 4 — RPG Systems

- prototype dialogue data;
- prototype quest state;
- add simple inventory;
- add item data;
- prototype save/load;
- introduce selected player-choice consequences.

23.5 Phase 5 — Vertical Slice

A first meaningful vertical slice should contain:

- one polished small location;
- one or more NPCs;
- one combat encounter;
- one short quest;
- dialogue;
- one meaningful choice;
- basic UI;
- working save/load if available by that stage;
- representative art and atmosphere.

24 Prototype Acceptance Criteria

The current prototype should be considered stable enough to move into a broader gameplay phase when:

1. movement behaves consistently in all four directions;
2. idle animation always matches the last facing direction;
3. attack state starts and ends predictably;
4. attack animations do not permanently lock player control;
5. player code remains understandable enough to extend safely;
6. project scenes open without missing critical resources;
7. the repository does not contain unnecessary editor-generated files;
8. public third-party assets have acceptable licences;
9. README and technical documentation accurately describe the implementation;
10. the current build can be demonstrated without requiring manual code fixes.

25 Long-Term Development Opportunities

Possible future systems include:

- reusable state machines;
- data-driven weapons and items;
- faction reputation;
- region-specific NPC behaviour;
- quest scripting tools;
- dialogue editors or external dialogue data;
- advanced save data;
- environmental fire or damage systems;
- richer enemy AI;
- accessibility settings;
- controller support;
- localisation;
- performance profiling for larger maps;
- automated testing for selected gameplay logic.

These features remain outside the immediate prototype scope unless a concrete development need appears.

26 Learning Objectives

VEK 0.4 is also a practical learning project.

The project is intended to build experience in:

- Godot Engine;
- GDScript;
- gameplay programming;
- object-oriented and state-based game logic;
- 2D animation;
- scene organisation;
- collision systems;
- game design;
- level design;
- debugging;
- Git-based project history;
- maintaining a larger codebase over time.

The project is therefore evaluated not only by how quickly features are added, but also by how much of the underlying implementation is understood.

27 Maintenance

Routine project maintenance should include:

- reviewing `.gitignore`;
- checking asset licences;
- removing temporary files;
- keeping documentation aligned with the actual prototype;
- testing the project after Godot updates;
- periodically reviewing script structure;
- keeping commit history understandable;
- backing up original artwork and source files;
- checking broken references after scene or asset reorganisations.

28 Conclusion

VEK 0.4 is an early-stage independent RPG project built around an existing fictional world rather than a setting created only for a programming exercise.

The current implementation is intentionally small. Its real foundation consists of player movement, directional animation, input handling, an early attack state and the first pieces of game structure.

The larger vision includes exploration, combat, narrative systems, dialogue, quests, world interaction and consequences, but those systems will be added gradually rather than presented as finished before they exist.

The project is meant to grow alongside the developer's technical ability. Its long-term value comes from combining practical game-development learning with a world that has been developed for years and has enough identity to support a much larger game.

End of VEK 0.4 game development documentation